

PROJECT REPORT: PYTHON PYQT5 STOPWATCH

Course: B.Tech Computer Science and Engineering (CSE)
Academic Year: 2026-2027

1. Project Overview

This project documents the development, debugging, and optimization of a desktop-based **Stopwatch Application** built using Python and the PyQt5 GUI framework. Dubbed '*Za Warudo Stopwatch*', the application provides precision time-tracking capabilities including start, stop, and reset configurations, packaged within an intuitive user interface.

2. Technical Specifications & Architecture

The software architecture follows an Object-Oriented Programming (OOP) paradigm utilizing event-driven components supplied by PyQt5. Key modules include:

Component	Description / Functionality
QApplication	Manages application-wide control flow, settings, and main event loop initialize routine.
QWidget	Serves as the base user-interface class from which the custom window layout inherits.
QTimer	Triggers cyclical timeout signals precisely mapped to 10-millisecond periods.
QTime	Maintains clock time parameters and arithmetic internal state representation.

3. Debugging Profile & Environment Resolutions

During implementation, several operational hurdles were systematically analyzed and resolved:

- Class Instance Name Overwrite Conflict:** The lowercase constructor invocation `stopwatch = stopwatch()` explicitly overrode the class definition context. This was refactored using proper CamelCase guidelines (Stopwatch class instantiated as a distinct `stopwatch_app` variable).
- Environment Variable Path Isolation:** Systems missing active terminal global path assignments encountered '*python/pip not recognized*' prompts. These were successfully resolved by utilizing the global Windows Python Launcher pipeline (`py -m pip install PyQt5`) or explicitly passing local path routing variables.

4. Source Code Blueprint

```
import sys
from PyQt5.QtWidgets import QApplication, QWidget, QLabel, QPushButton, QVBoxLayout, QHBoxLayout
from PyQt5.QtCore import QTimer, QTime, Qt

class Stopwatch(QWidget):
    def __init__(self):
        super().__init__()
        self.time = QTime(0, 0, 0, 0)
        self.time_label = QLabel("00:00:00.00", self)
        self.start_button = QPushButton("start", self)
        self.stop_button = QPushButton("stop", self)
        self.reset_button = QPushButton("reset", self)
        self.timer = QTimer(self)
        self.initUI()

    def initUI(self):
        self.setWindowTitle("za warudo stopwatch")
        vbox = QVBoxLayout()
        vbox.addWidget(self.time_label)
        self.time_label.setAlignment(Qt.AlignCenter)

        hbox = QHBoxLayout()
        hbox.addWidget(self.start_button)
        hbox.addWidget(self.stop_button)
        hbox.addWidget(self.reset_button)
        vbox.addLayout(hbox)
        self.setLayout(vbox)

        self.setStyleSheet("""
            QPushButton, QLabel{ padding: 20px; font-family: calibri;}
            QPushButton{font-size: 50px;}
            QLabel{font-size: 120px; background-color: red}
        """)

        self.start_button.clicked.connect(self.start)
        self.stop_button.clicked.connect(self.stop)
        self.reset_button.clicked.connect(self.reset)
        self.timer.timeout.connect(self.update_display)

    def start(self): self.timer.start(10)
    def stop(self): self.timer.stop()
    def reset(self):
        self.timer.stop()
        self.time = QTime(0, 0, 0, 0)
        self.time_label.setText("00:00:00.00")

    def format_time(self, time):
        hours, minutes = time.hour(), time.minute()
        seconds, milliseconds = time.second(), time.msec() // 10
        return f"{hours:02}:{minutes:02}:{seconds:02}.{milliseconds:02}"

    def update_display(self):
        self.time = self.time.addMSecs(10)
        self.time_label.setText(self.format_time(self.time))

if __name__ == '__main__':
    app = QApplication(sys.argv)
```

```
stopwatch_app = Stopwatch()  
stopwatch_app.show()  
sys.exit(app.exec_())
```

5. Operational Conclusion

The resulting implementation yields a highly responsive stopwatch tracking system utility. By using event-driven timeout updates instead of structural loops, the framework operates efficiently without dropping frames or bottlenecking the underlying system resources.